

Laboratorio di Reti e Sistemi Distribuiti

Progetto finale - Server FTP concorrente in C

Aurora Fabio

Università degli studi di Messina

`aurora.fabio@studenti.unime.it`

professore Roberto Marino

30 aprile 2026

- 1 Descrizione del problema
- 2 Stato dell'arte
- 3 Metodologia ed Implementazione
- 4 Risultati
- 5 Conclusioni e sviluppi futuri

- Sviluppo di un **server FTP anonimo concorrente in C** che opera in modalità passiva
- Gestione **multiprocessing** di client multipli
- Implementazione delle richieste `CWD`, `LIST`, `RETR`, `STOR`, `QUIT` in modalità anonima
- Sincronizzazione con **semaforo POSIX**

Protocollo FTP - File Transfer Protocol

Il protocollo FTP, definito nella RFC 959, si basa su un'architettura **client-server**:

- il **client** fornisce i comandi per connettersi, visualizzare e trasferire i file al server
- il **server** è un sistema che ospita i file, in attesa delle richieste del client
- **out-of-band**: i dati viaggiano sul canale dei dati mentre i comandi sulla porta 21.

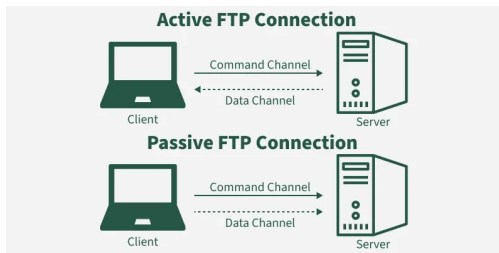


Figura: Connessione attiva/passiva

Protocollo FTP - File Transfer Protocol

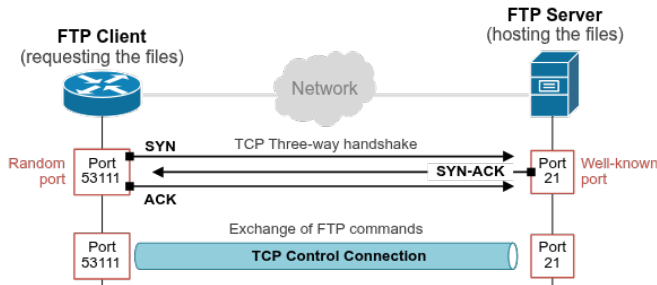


Figura: FTP

La modalità di trasmissione dei dati che è stata applicata è la modalità di **flusso**, in cui i dati vengono inoltrati come un flusso continuo di byte ed a gestirne la frammentazione è il TCP. AL termine di ogni trasmissione, connessione viene chiusa.

In ambiente POSIX, la concorrenza può essere gestita in vari modi:

- **Modello multi-processo:** utilizza la system call `fork()` e isola la memoria legata ad ogni processo
- **Modello multi-thread:** memoria condivisa tra i thread e richiede la sincronizzazione (`mutex`, `semafori`)
- **I/O Multiplexing:** utilizza `select()`, `poll()`, `epoll()` per monitorare più file descriptor e ottimizzare l'uso della CPU per le connessioni inattive

Nel protocollo FTP, basato sulla RFC 959, i dati, i comandi e le credenziali viaggiano in chiaro. Ad oggi, l'FTP può essere incapsulato in protocolli sicuri:

- **FTPS**: aggiunge il livello TLS/SSL all'architettura di base
- **SFTP**: utilizza il protocollo SSH, con un unico canale di comunicazione cifrato

Per la gestione di client simultanei, ho implementato un'architettura **multi-processo**:

Processi

Processo padre

- Inizializza il socket di ascolto (`socket()`, `bind()`, `listen()`)
- Attende nuove connessioni tramite `accept()`
- Per ogni client, delega la gestione al processo figlio tramite `fork()`

Processo figlio

- Eredita il socket connesso al client
- Esegue la funzione `handle_session()`
- In caso di interruzione di un singolo client, garantisce stabilità globale operando in spazi di memoria isolati

Pseudocodice `accept()` loop con `fork()`

LOOP:

```
client ← accept()

IF active_clients ≥
  MAX_CLIENTS:
  invia 421
  rifiuta connessione

pid ← fork()

IF pid == 0: // figlio
  active_clients++;
  handle_session(client)
  active_clients--;

IF pid > 0: // padre
  torna al LOOP
```

Concorrenza

- controllo: il padre verifica il limite `MAX_CLIENTS` per risparmiare risorse prima di invocare la `fork()`
- gestione del contatore: il figlio incrementa/decrementa il contatore condiviso per evitare race condition
- delega: dopo aver creato il figlio, il processo padre torna in ascolto sulla `accept()`

Per evitare che client inattivi esauriscano le risorse del sistema tenendo occupati i processi figli all'infinito, non ho usato la chiamata bloccante `recv()`. Per questo motivo, ho implementato un meccanismo di **I/O Multiplexing** tramite la system call `select()`

Timeout

- **Monitoraggio:** la `select()` monitora il file descriptor del client.
- **Timeout:** limite di **30 secondi**.
- **Risoluzione:** se il client non invia nulla entro la scadenza, la funzione ritorna 0, il server invia il codice 421 (Timeout), chiude i socket e termina il processo figlio, rilasciando la memoria.

Sincronizzazione e gestione della Race Condition

In un'architettura multi-processo, i figli operano in spazi di memoria isolati. Tuttavia, condividono l'accesso a risorse fisiche del sistema, come il file di log condiviso (`/tmp/server.log`).

Il problema: la Race Condition

Se due processi figli tentano di leggere e modificare `active_clients` contemporaneamente, potrebbero ottenere lo stesso valore e accettare entrambi una connessione, superando il limite `MAX_CLIENTS`

La soluzione: Semaforo con Mutua Esclusione

Il semaforo protegge l'accesso al contatore `active_clients` nella memoria condivisa, garantendo che un solo processo alla volta possa leggere o modificare il valore

- il semaforo risiede in un'area di memoria condivisa creata con `mmap()`
- è inizializzato con `MAP_SHARED` per essere visibile a processi distinti

Pseudocodice Semaforo Binario

```
// Inizializzazione (padre):  
sem <- 1  
  
// Lettura protetta (figli):  
sem_wait(sem)  
  // sem <- sem - 1 (blocco)  
  
clients <- *active_clients  
  
sem_post(sem)  
  // sem <- sem + 1 (sblocco)  
  
// Controllo del carico  
IF clients >= MAX_CLIENTS:  
  rifiuta connessione  
ELSE:  
  accetta connessione
```

Logica di Controllo

- **Memoria Condivisa:**
active_clients risiede in un'area comune a tutti i processi.
- **Mutua Esclusione:** per la sincronizzazione dei processi
- **Prevenzione Sovraccarico:**
evita che connessioni simultanee aggirino il controllo.

Per aderire al principio DRY (Don't Repeat Yourself), le risposte del server sono state rese astratte.

```
ftp_reply()
```

- formatta il codice a 3 cifre e il testo secondo lo standard FTP
- invia la stringa sul socket di controllo tramite `send()`

Tutto il traffico di controllo viene registrato tramite la funzione `cmd_log()` e salvato nel percorso `/tmp/server.log`, che è una directory volatile.

Gestione del File System e trasferimento dei dati

CWD/PWD

- il cambio di directory diventa effettivo con la system call `chdir()`
- la funzione `getcwd()` estrae il percorso assoluto affinché venga aggiornata la variabile di stato `cwd`

LIST

- `opendir()` e `readdir()` gestiscono l'apertura e la scansione del contenuto della directory
- `struct dirent` formatta e invia il nome dei file sul canale dati

RETR

- apertura in lettura binaria ("`rb`") in blocchi da 1024 byte con `fread()`
- invio continuo sul canale dati tramite `send()`

STOR

- apertura in scrittura binaria ("`wb`") in blocchi da 1024 byte con `fwrite()`
- ricezione dal canale dati tramite `recv()`

Pseudocodice Sessione FTP

```
invia 220 Benvenuto al server di  
Aurora Fabio
```

LOOP:

```
comando ← recv()  
IF USER → 331  
IF PASS → 230, logged_in=1  
IF PASV → apri connessione  
           passiva  
IF LIST → invia lista file  
IF RETR → invia file  
IF STOR → ricevi file  
IF QUIT → 221, esci
```

Pseudocodice PASV

```
apertura socket su porta casuale  
porta ← getsockname()  
p1 ← porta / 256  
p2 ← porta % 256  
  
invia "227 (127,0,0,1,p1,p2)"  
data_fd ← accept()
```

Comando	Client	Descrizione Funzionale
CWD	cd	Navigazione directory tramite system call <code>chdir()</code> .
LIST	ls	Scansione directory tramite <code>opendir()</code> e <code>readdir()</code> .
RETR	get	Trasferimento file dal Server al Client (modalità "rb").
STOR	put	Ricezione file dal Client al Server (modalità "wb").
QUIT	bye / quit	Chiusura sessione e rilascio risorse IPC.

Obiettivi raggiunti

- **modalità passiva**
- **gestione della concorrenza tra processi**
- **logging**

```
[10:14:44] 220 Benvenuto al server FTP di Aurora Fabio  
[10:14:44] USER anonymous  
[10:14:44] 331 Password richiesta  
[10:14:44] PASS password  
[10:14:44] 230 Login effettuato  
[10:14:44] SYST  
[10:14:44] 502 Comando sconosciuto!  
[10:14:44] FEAT  
[10:14:44] 502 Comando sconosciuto!  
[10:15:14] 421 Client disconnesso per timeout della connessione
```

Figura: /tmp/server.log

Test di carico

Per verificare la stabilità dell'architettura multi-processo, ho simulato un ambiente ad alta concorrenza tramite uno script **Bash** (test_carico.sh)

```
227 Accesso modalità passiva (127,0,0,1,186,225)
227 Accesso modalità passiva (127,0,0,1,152,121)
150 Invio file
150 Invio file
   6    102.79 KiB/s
   6    91.55 KiB/s
226 Trasferimento completato
226 Trasferimento completato
6 bytes received in 00:00 (0.14 KiB/s)
6 bytes received in 00:00 (0.14 KiB/s)
local: test.txt remote: test.txt
local: test.txt remote: test.txt
227 Accesso modalità passiva (127,0,0,1,147,39)
227 Accesso modalità passiva (127,0,0,1,128,177)
150 Pronto alla ricezione
150 Pronto alla ricezione
 0% |                                     |   0      0.00 KiB/s  --:-- ETA2
26 Trasferimento completato
100% |*****|   6    154.19 KiB/s  --:-- ETA
6 bytes received in 00:00 (0.14 KiB/s)
100% |*****|   6    122.07 KiB/s  --:-- ETA
```

Figura: Test di carico

Risultati della scalabilità

- Tutte le richieste sono state instradate dal processo padre tramite la system call `fork()`
- non è stato rilevato nessun deadlock o perdita di connessione
- l'area di memoria è stata correttamente isolata

canale di controllo (PORTA 21)

- **Three-way handshake TCP**
- **plain text** dei comandi e dei codici

canale dati e (PASV)

- porte effimere
- chiusura connessione TCP

No.	Time	Source	Destination	Protcl	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	41782 → 21 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 SACK_PERM TSval=1925950610 TSecr=0 WS=4
2	0.000048051	127.0.0.1	127.0.0.1	TCP	74	21 → 41782 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=1925950610 TSecr=1925950610 WS=1024
3	0.000075429	127.0.0.1	127.0.0.1	TCP	66	41782 → 21 [ACK] Seq=1 Ack=1 Win=130992 Len=0 TSval=1925950610 TSecr=1925950610
4	0.000620472	127.0.0.1	127.0.0.1	FTP	111	Response: 220 Benvenuto al server FTP di Aurora Fabio
5	0.000646732	127.0.0.1	127.0.0.1	TCP	66	41782 → 21 [ACK] Seq=1 Ack=46 Win=130948 Len=0 TSval=1925950610 TSecr=1925950610
6	5.019126713	127.0.0.1	127.0.0.1	FTP	72	Request: EPSV
7	5.019161494	127.0.0.1	127.0.0.1	TCP	66	21 → 41782 [ACK] Seq=46 Ack=7 Win=65536 Len=0 TSval=1925955629 TSecr=1925955629
8	5.019328555	127.0.0.1	127.0.0.1	FTP	90	Response: 530 Effettua l'accesso
9	5.019349996	127.0.0.1	127.0.0.1	TCP	66	41782 → 21 [ACK] Seq=7 Ack=70 Win=130944 Len=0 TSval=1925955629 TSecr=1925955629
10	5.019473476	127.0.0.1	127.0.0.1	FTP	72	Request: PASV
11	5.019634111	127.0.0.1	127.0.0.1	FTP	116	Response: 227 Accesso modalità passiva (127,0,0,1,128,67)

Figura: Modalità passiva

Conclusioni e sviluppi futuri

È stato sviluppato un server FTP basato sugli standard della **RFC 959**

Obiettivi raggiunti

- **architettura robusta**: il server gestisce correttamente la concorrenza multi-processo
- **ottimizzazione delle risorse**: l'**I/O Multiplexing** previene il blocco da parte dei client inattivi
- **affidabilità**: il test di carico e l'analisi di rete hanno confermato la stabilità del sistema sotto stress e la corretta implementazione del paradigma out-of-band e della modalità passiva

Sviluppi futuri

- l'uso dei thread al posto dei processi per le loro prestazioni
- l'implementazione di un'autenticazione reale e non più anonima
- l'ampliamento dei comandi FTP, come l'eliminazione (DELE) o la creazione (MKD) delle directory
- utilizzo di chroot() per impedire al client di navigare all'esterno della cartella condivisa

Grazie per l'attenzione!

Aurora Fabio 558520
aurora.fabio@studenti.unime.it